

1. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

---

```
abhinav@Abhinav:~$ mkdir PROCESSID
abhinav@Abhinav:~$ nano PROCESSID.c
abhinav@Abhinav:~$ nano PROCESSID.c
abhinav@Abhinav:~$ gcc PROCESSID.c -o program
abhinav@Abhinav:~$ ./program
My Process ID (PID) = 654
```

OUTPUTS: abhinav@Abhinav:~\$

```
abhinav@Abhinav:~$ nano CHILDPROCESS.c
abhinav@Abhinav:~$ gcc CHILDPROCESS.c -o program
abhinav@Abhinav:~$ ./program
Parent Process
PID = 919
Child Process
PID = 920
```

```
PID = 920
abhinav@Abhinav:~$ mkdir PARENTPID
abhinav@Abhinav:~$ nano PARENTPID.c
abhinav@Abhinav:~$ gcc PARENTPID.c -o program
abhinav@Abhinav:~$ ./program
Child Process Executing...
Child Finished
Parent Process Continues
abhinav@Abhinav:~$
```

```
Parent Process Continues
abhinav@Abhinav:~$ gcc MULTIPLECHILD.c -o program
abhinav@Abhinav:~$ ./program
Child 1 PID = 1081
Child 2 PID = 1082
Child 3 PID = 1083
abhinav@Abhinav:~$
```